

Subject Alternative Names (SANs) for the Kubernetes API Server

Why they matter and how to configure them with kubeadm

What a SAN is

A **Subject Alternative Name** is the field in the API server's TLS certificate that lists every hostname and IP address the certificate is valid for. When a client connects to `https://<address>:6443`, it checks that the address appears in the SAN list. If it does not, the TLS handshake fails with an error such as:

```
x509: certificate is valid for 10.96.0.1, 192.168.1.10, not 203.0.113.5
```

By default, kubeadm includes the node's hostname, its advertise IP, the in-cluster service IP (for example `10.96.0.1`), and names such as `kubernetes.default.svc`.

Benefits of adding extra SANs

Reaching the API server by more than one address. If you also access the API server through a public IP, a DNS name such as `k8s.example.com`, or a VPN address, each of those must be a SAN.

High availability. With several control-plane nodes behind a load balancer, clients connect to the load balancer's address rather than to any single node. That VIP or DNS name must be in the SANs of every API server certificate. This is the most common reason to set them.

Changing infrastructure without re-issuing certificates. If kubeconfigs point at a stable DNS name that is included as a SAN, you can later move nodes, change IPs, or replace load balancers by updating DNS alone.

Security without workarounds. Without correct SANs, people often fall back to `insecure-skip-tls-verify: true`, which removes protection against man-in-the-middle attacks. Proper SANs keep full certificate verification working from every access path.

Tooling and automation. CI/CD systems, monitoring, GitOps controllers, and external services that connect from outside the cluster all need an address that validates.

Setting SANs with kubeadm

Add SANs to the `ClusterConfiguration` before running `kubeadm init`:

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
controlPlaneEndpoint: "k8s.example.com:6443"
apiServer:
  certSANS:
    - "k8s.example.com"
    - "203.0.113.5"
    - "10.0.0.100"      # load balancer VIP
```

```
sudo kubeadm init --config kubeadm-config.yaml
```

Alternatively, set them inline:

```
sudo kubeadm init --apiserver-cert-extra-sans=k8s.example.com,203.0.113.5
```

Adding SANs to an existing cluster

Update certSANS in the kubeadm config, then regenerate the certificate:

```
sudo mv /etc/kubernetes/pki/apiserver.{crt,key} /tmp/
sudo kubeadm init phase certs apiserver --config kubeadm-config.yaml
```

Then restart the API server static pod. The usual way is to move its manifest out of /etc/kubernetes/manifests/ and back again. Verify the result with:

```
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -noout -text \
| grep -A1 "Subject Alternative Name"
```

Practical tip

Set controlPlaneEndpoint to a DNS name and include it as a SAN from day one, even on a single-node cluster. It makes converting to a highly available control plane much easier later.